

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

10. Calidad e Integridad de Datos	1
10.1. Calidad de Datos	2
10.1.1. Validación de Datos	3
10.1.2. Gestión de la Calidad de Datos	4
10.2. Integridad de Datos	7
10.3. Sumas de Verificación	9
10.3.1. Sumas de Verificación	9
10.3.2. Sumas de verificación en HDFS	11
10.4. Movimientos de datos con DistCP	12
10.4.1. Arquitectura	12
10.4.2. Funcionamiento básico	13
10.4.3. Funcionamiento Mejorado	13
10.4.4. Uso	15

Listado de acrónimos

ETL	Extraction Transformation and Load
HDFS	Hadoop Distributed File System

Capítulo 10

Calidad e Integridad de Datos

En este capítulo se presentan temas relacionados con la calidad de datos, la integridad de los datos y los movimientos de datos entre clústers.

La **calidad de los datos** suele concebirse como un concepto multidimensional definido como "aptitud para su uso". La calidad de los datos puede depender de varios factores (dimensiones o características) como la completitud, la coherencia, la disponibilidad, etc. La evaluación de la calidad de los datos implica la medición de las dimensiones o criterios de calidad que son relevantes para cada caso de uso. Una métrica o medida es un procedimiento para medir una dimensión de calidad de datos con la ayuda de una herramienta. Estas métricas son heurísticas que están diseñados para adaptarse a una situación de evaluación específica [WZL06].

El concepto de **integridad de datos** refiere a la precisión, la coherencia y la integralidad los datos de una organización. La integridad de un dato alude a un atributo o cualidad inherente a la información que se considera exacta, completa, homogénea, sólida, coherente y confiable, con base en la intención de los creadores de las bases de datos.

Dentro de un entorno Big Data esto es especialmente crítico, dado que a mayor volumen de datos corresponde un mayor peligro relacionado con la integridad de los mismos, por lo que será necesario implementar mecanismos avanzados de control como las sumas de verificación, asumiendo la sobrecarga que conlleva a las operaciones de los sistemas de archivos.

Por último, se tratará el tema del movimiento de datos entre clústers, para ello se utilizará el comando copia distribuida, `distcp`. Este comando es una utilidad general para copiar grandes conjuntos de datos entre sistemas de archivos distribuidos dentro y entre clústeres.

10.1. Calidad de Datos

Un lago de datos o *Data Lake* es un pool para almacenar datos no relacionales sin transformación, lo cual puede incluir datos transaccionales y de ventas, archivos de *log*, encuestas de clientes, comentarios de redes sociales, etc. El objetivo de este almacenamiento es posteriormente procesarlos y analizarlos con el fin de ayudar a la toma de decisiones en la organización.

La utilización del ecosistema Hadoop como infraestructura básica del *Data Lake* está bastante extendida ya que es especialmente adecuada para gestionar datos que se generan gran velocidad y con una gran variedad, además permite un bajo coste de la infraestructura. Empresas como la banca, el comercio minorista, la medicina, la aeronáutica, etc., utilizan estos datos consolidados para desarrollar análisis históricos, predictivos, prescriptivos e incluso análisis basados en el aprendizaje profundo o de investigación.

Los elementos clave de un *Data Lake* pueden considerarse los siguientes[PP15]:

- **Data Quality** (Calidad de Datos): Medida para evaluar la corrección de los conjuntos de big data que se manejan en la infraestructura.
- **Data Profiling** (Perfilado de Datos): Evaluación estadística de los valores de los datos dentro de un conjunto de consistencia, singularidad y lógica según las reglas del negocio.
- **Data Governance** (Gobierno de los datos): Gestión global de la disponibilidad, usabilidad, integridad y seguridad de los datos.
- **Data Lineage** (Alineamiento de los datos): metadatos que definen de dónde proceden los datos y cómo se calcularon



Figura 10.1: Elementos de Calidad de Datos del *Data Lake*

Estudios recientes han demostrado que es habitual que la calidad de los datos agregados en el *Data Lake* diste mucho de unas características aceptables. La mala calidad de los datos puede suponer una pérdida tangible para la empresa en términos de reducción de ingresos, servicios ineficientes, mayor coste de mantenimiento, etc. Por eso, la validación de datos es un proceso crítico que permitirá reconocer y mejorar la calidad de los datos.

10.1.1. Validación de Datos

El proceso de validación de datos consiste en garantizar la validez de los datos y verificar que los datos son fiables, precisos y significativos. Estadísticamente ha quedado demostrado que más de la mitad del tiempo invertido en proyectos de Big Data se destina a la limpieza y preparación de los datos [CZ15].

Los procesos de validación de datos son los siguientes: recogida de datos, la limpieza de datos, la transformación de datos carga de datos e informe de resultados.

1. **Data Collection** (Captura de Datos): Los datos se recogen de diversas fuentes como correos electrónicos, bases de datos, archivos CSV , excels, sistema de archivos, etc.
2. **Data Loading** (Carga de Datos): Se cargan los datos en un repositorio de big data, por ejemplo, un entorno Hadoop (HDFS) o una base de datos NoSQL. Dependiendo de los requisitos del cliente, la actualización de los datos extraídos se realiza con frecuencia una base diaria, semanal o mensual.
3. **Data Validation** (Validación de Datos): Se comprueba la calidad de los datos, por ejemplo, el tipo de datos, los formatos números de teléfono, valores numéricos, etc.
4. **Data Cleaning** (Limpieza de Datos): Se detectan los datos incompletos, incorrectos, inexactos o irrelevantes en los conjuntos de datos y se propone una solución a cada una de las situaciones encontradas.
5. **Data Transformation** (Transformación de Datos): Proceso por el que se convierten los datos originales al formato o estructura necesario para el negocio.
6. **Data Analytics** (Análítica de Datos): Elaborar informes que van desde el propio proceso de validación de los datos (consistencia, integridad, etc.) y tareas de inteligencia de negocio en general.

Dentro de la infraestructura Big Data se hace necesario por tanto el desarrollar procesos que analicen los datos que han sido capturados mediante cargas por lotes o incrementales y almacenados en HDFS para su explotación por las diversas herramientas del Ecosistema. El sistema de calidad de datos debe permitir el análisis de

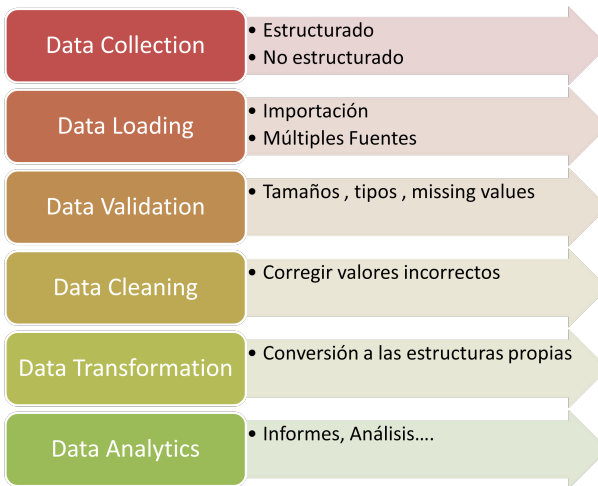


Figura 10.2: Proceso de validación de los datos

gran cantidad de información (del orden de TB's) en tiempos reducidos. Funcionalmente debe ser capaz de realizar tanto las validaciones básicas (valores perdidos, tipos de datos y expresiones regulares de formato) como validaciones adaptadas a las reglas de negocio generar los informes al efecto que pueden incluir: las anomalías detectadas, el tipo de incorrección, la localización exacta de la misma (archivo y línea), estadísticas diversas sobre la calidad de los datos etc.

Una dimensión muy a tener en cuenta es la obsolescencia de los datos, es por ello que las tareas de validación deben programarse y permitir inferir la vigencia de los datos durante un período de tiempo.

10.1.2. Gestión de la Calidad de Datos

El proceso de Gestión de la Calidad de los Datos debe realizar tareas de validación de datos (Data Validation), perfilado de datos (Data Profiling), análisis de obsolescencia (Timeliness Analysis), limpieza de datos, etc. Dentro de un entorno Big Data basado en Hadoop estos procesos estarán basados en trabajos MapReduce que se ocuparán de analizar los archivos importados por lotes o de forma incremental y almacenados en HDFS.

Será necesario analizar TB's de de datos en un tiempo reducido y también encontrar anomalías utilizando categorías genéricas de validaciones: Null, Expresiones Regulares (Regex), tipos de datos, etc. Además debe ser capaz de generar los informes necesarios que muestren los problemas encontrados en los datos almacenados en el sistema HDFS.

Data Validation

Se deberá implementar un proceso de validación de datos sobre HDFS con el fin de realizar las comprobaciones necesarias sobre los datos. Estas comprobaciones, que deberán programarse periódicamente, incluirán la búsqueda de anomalías y el cumplimiento de las reglas marcadas por la organización. Estas reglas podrán ser para el control de nulos, verificación de tipos de datos y/o expresiones regulares. Además será necesario generar informes que ofrezcan detalles sobre las situaciones encontradas como tipo de error, nombre de archivos, campos más afectados, etc.



La detección de valores atípicos se refiere a la actividad de detectar valores atípicos y es una tarea cuantitativa de detección de errores. Aunque la definición exacta de un valor atípico depende de la aplicación, hay algunas definiciones comúnmente utilizadas, como *un valor atípico es una observación que se desvía tanto de otras observaciones que despiertan la sospecha de que se ha generado por un mecanismo diferente*

Data Cleaning

La limpieza de datos se utiliza para referirse a todo tipo de tareas y actividades para detectar y reparar errores en los datos.

En este proceso no sólo se deben limpiar los datos de forma automatizada (verificación de nulos, verificación de tipo de datos y expresiones regulares) es necesario proporcionar la capacidad de almacenar los datos inconsistentes para propósitos futuros (*Dead Letter Channel*). Por último se deben proporcionar informes que muestren las estadísticas de archivos limpios y sucios, etc.



El **Dead Letter Channel** (DLC) es un patrón sobre cómo los sistemas de mensajería pueden manejar los mensajes que no pueden ser entregados al destinatario. Debido a fallos del sistema/red o a fallos en destino, a veces los mensajes no se entregan al objetivo. En estos casos, el sistema de mensajería puede entregar el mensaje a un DLC. Diferentes mecanismos implementados en el DLC se encargan de entregar el "mensaje muerto" al objetivo. Uno de los métodos consiste en reintentar periódicamente el envío del mensaje al destinatario durante un periodo de tiempo definido. La persistencia del mensaje muerto es otra opción, que asegura que los mensajes muertos se entreguen a los receptores una vez que el sistema se reinicie, incluso si el sistema de mensajería falla.

Data Profiling

El perfilado de datos o *Data Profiling* es el conjunto de actividades y procesos para determinar los metadatos sobre un determinado conjunto de datos[AGN17]. Los metadatos pueden ir desde simples estadísticas, como el número de valores nulos y valores distintos en una columna, su tipo de datos, o los patrones más frecuentes de sus valores, hasta complejas dependencias entre valores y columnas. Los metadatos que son más difíciles de calcular implican a varias columnas, como como las dependencias de inclusión o las dependencias funcionales.

Al conjunto de resultados de estas tareas se denomina perfil de datos o perfil de base de datos. Se considera al perfilado de datos como la actividad de crear resúmenes pequeños pero informativos de una base de datos. A diferencia de la minería de datos de datos, el enfoque de la creación de perfiles de datos suele ser la estructura de un conjunto de datos. Así, la creación de perfiles de datos genera conocimientos sobre las columnas de un conjunto de datos, mientras que la minería de datos se centra en los registros.

La elaboración de perfiles de datos abarca una amplia gama de métodos para examinar conjuntos de datos y producir metadatos. Al igual que muchas tareas de gestión de datos, la elaboración de perfiles de datos se enfrenta a tres retos:

- la gestión de la entrada,
- realizar eficazmente el cálculo,
- la gestión del resultado.

Independientemente de los problemas comunes que presentan los datos, el primer reto se refiere al problema de especificar el resultado esperado, es decir, determinar qué tareas de elaboración de perfiles deben ejecutarse en qué partes de los datos. De hecho, muchas herramientas requieren una especificación precisa de lo que se debe inspeccionar. Otros enfoques son más abiertos y realizan una más amplia de tareas, descubriendo todos los metadatos automáticamente.

Para realizar esta labor se necesita realizar un análisis profundo de los datos. Alguna de la información que se puede captar durante el perfilado de datos es la siguiente: el tipo de datos, la longitud valores discretos, unicidad, aparición de valores nulos, patrones de cadena típicos puede utilizarse para descubrir problemas como valores ilegales, errores ortográficos, valores que faltan, variaciones en la representación de valores, y duplicados.

Los datos de HDFS también pueden ser perfilados sobre un conjunto de reglas o sin ellas para obtener algunas obtener algunas ideas rápidas sobre los datos ingestados.

10.2. Integridad de Datos

La integridad de los datos es la garantía de exactitud y coherencia de los datos a lo largo de su ciclo de vida (desde que se registran hasta que se destruyen). La integridad de los datos implica que se han registrado los datos tal y como se pretendía y que no se han modificado involuntariamente a lo largo de su ciclo de vida. El concepto es sencillo, pero la práctica no lo es. La integridad de los datos es un componente crítico para crear o diseñar cualquier sistema de software que almacene o mueva datos[LEHO16].

La integridad de los datos es un aspecto muy relevante dentro de un entorno Big Data dado que la mayoría de las decisiones críticas se basan en los datos de una organización. Con una buena integridad de los datos. Si no se dispone de datos con integridad y calidad la mayoría de las preguntas cruciales para el progreso de la organización quedarán sin respuesta o ésta será imprecisa.

La integridad de los datos proporciona protección contra la alteración de los datos por parte de un usuario no autorizado de forma no autorizada. Algunas de las razones de problemas con la integridad de los datos son las siguientes:

Los errores de hardware, los errores del usuario, errores de software, o intrusos son las principales razones de los problemas de integridad de los datos [SJR15]

- **Error humano:** Afectan la integridad la información incorrecta, datos no válidos, duplicación o eliminación de datos sin seguir el protocolo adecuado, errores en los procedimientos de protección de la información.
- **Errores de transferencia:** al transferir datos de una ubicación a otra o se dispone de un dato en destino que no está en la tabla de origen.
- **Ataques:** Los ataques de manipulación de datos, los ataques a la relación de confianza, ataques de secuestro de sesión, etc. son los más conocidos contra la integridad de los datos [PA15].
- **Hardware:** Problemas de funcionamiento de un dispositivo o daños inesperados de los ordenadores o servidores, son fallos que indican que el hardware transmite datos de forma incorrecta o incompleta, limita o elimina el acceso a los mismos o complica el uso de la información.

La integridad puede mantenerse mediante los siguientes mecanismos:

- **Data Provenance:** Este concepto está relacionado con la información sobre el proceso de creación, así como sobre las fuentes de datos a través de las cuales se transforman. En Big Data, es común que los datos de diferentes fuentes y orígenes se integren (fusión de datos). Sin embargo, si una o varias fuentes de datos combinadas son de baja calidad es difícil filtrarlos después de haber sido combinados y agregados. La procedencia de los datos se basa en la identificación clara de cada unidad de datos y permite eliminar los datos de fuentes no fiables. Supone la comprobación de todos los estados de los datos desde estado inicial hasta el estado actual. La depuración, la seguridad

y los modelos de confianza son varias aplicaciones de este concepto. Sin información sobre la procedencia de los datos, el usuario nunca llega a saber de dónde proceden los datos y qué transformaciones se han aplicado a los mismos.

- *Data trustworthiness*: Para lograr la confiabilidad de los datos en un entorno Big Data las organizaciones deben implementar y automatizar procesos para auditar, evaluar y limpiar sus datos. Esto debe ir acompañado por una infraestructura de datos que tenga en cuenta los procesos humanos junto con el software. Es necesario crear una cultura centrada en los datos que funcione en conjunto con la automatización de la calidad de los datos. Será necesario por lo tanto implementar tanto medidas preventivas para identificar y resolver los problemas de los datos en toda la organización, como tratamientos eficaces para combinar procesos automatizados y manuales para resolver los problemas que se pueden presentar. La infraestructura necesaria aprovechará los conocimientos de los usuarios del negocio para limpiar los datos, así como herramientas sofisticadas que permitan a los ingenieros de datos realizar operaciones complejas sin necesidad de conocimientos de codificación. La confiabilidad de los datos se podría medir considerando dimensiones como la exhaustividad, transparencia, oportunidad y certificación de los datos. Con un acceso abierto a datos completos, limpios y fiables, los usuarios finales pueden tomar decisiones mejores con la máxima confianza.
- *Data Loss*: Como recursos con valor los datos están sujetos a robo (puede ser peor que la pérdida, si la entidad que lo roba abusan del recurso), la indisponibilidad (no tan grave como la pérdida o la corrupción, pero no deja de ser un inconveniente), la corrupción o la pérdida permanente. Los datos se pierden o dejan de estar disponibles para su uso si el dispositivo de almacenamiento o el sitio anfitrión que contiene los datos se estropea o no responde. La indisponibilidad de los datos puede evitarse manteniendo múltiples copias de los datos en diferentes dispositivos/sitios o mediante la codificación distribuida. En el contexto de los big data, todos los aspectos del sistema (procesamiento, almacenamiento y comunicación) aumentarán su complejidad, lo que el diseñador del sistema a buscar métodos más avanzados para garantizar la disponibilidad y la fiabilidad.
- *Data Deduplication*: La detección de duplicados se refiere al proceso de identificar tuplas en una o más relaciones que se refieren a la misma entidad del mundo real. Este proceso suele implicar muchos pasos y opciones, incluido el diseño de métricas de similitud para evaluar la similitud de un par de registros, el entrenamiento de clasificadores para determinar si un par de registros son duplicados o no, la agrupación de todos los registros para obtener clusters de registros que representan la misma entidad del mundo real, consolidando los clusters de registros en representaciones únicas, diseñar estrategias de bloqueo o distribuidas para aumentar la escala del proceso de deduplicación y humanos para decidir si un par de registros son duplicados cuando las máquinas no están seguras.

10.3. Sumas de Verificación

Garantizar la integridad de los datos es una necesidad básica o en un entorno de procesamiento Big Data para lograr resultados precisos, de la misma forma que se aplica a cualquier operación de movimiento de datos con los sistemas tradicionales de almacenamiento de datos (RDBMS, Repositorio de Documentos, etc.) a través de diversas aplicaciones. El transporte de datos a través de la red, la transferencia de dispositivo a dispositivo, el proceso ETL, y muchos más.

En un ecosistema Big Data los datos se manejan de forma distribuida debido a su enorme volumen. Por lo tanto, lograr la integridad de los datos es un auténtico reto. HDFS ha sido concebido para almacenar cualquier tipo de datos de manera distribuida en forma de bloque de datos (descompone un enorme volumen de datos en un conjunto de bloques individuales) con compromiso de integridad de datos. Puede haber múltiples causas que generen bloques de datos corruptos en HDFS, empezando por la operación de I/O en el disco del sistema, el fallo de la red, etc.

10.3.1. Sumas de Verificación

Una suma de comprobación (checksum) es un número calculado a partir de un bloque de datos arbitrario. Este número puede utilizarse para detectar errores que puedan haberse introducido durante su transmisión o almacenamiento. La integridad de los datos puede comprobarse en cualquier momento posterior volviendo a calcular la suma de comprobación y comparándola con la almacenada. Si las sumas de comprobación coinciden, es probable que los datos no hayan sido alterados accidentalmente.

El procedimiento que produce la suma de comprobación a partir de los datos se denomina función de suma de comprobación o algoritmo de suma de comprobación. Un buen algoritmo de sumas de comprobación arrojará un resultado diferente con alta probabilidad cuando los datos se corrompan accidentalmente; si las sumas de comprobación coinciden, los datos tienen la misma alta probabilidad de estar libres de errores accidentales.

Las funciones de suma de comprobación se confunden a menudo con los hash o incluso con los cifrados. Todas están relacionadas, pero cada una tiene sus propios usos y está diseñada de forma diferente.

Existen varios tipos y versiones de algoritmos de verificación de datos que constituyen una práctica estándar recomendada para la detección de errores accidentales o intencionales acaecidos en ficheros de archivo. Los algoritmos criptográficos son los más utilizados para esta labor, pero presentan el problema de la colisión hash. Por una regla matemática llamada principio de encasillamiento, no se puede asignar de forma exclusiva cualquier trozo de datos posible a una huella digital más corta. Estadísticamente, hay múltiples datos posibles que tienen el mismo hash. Según los estándares actuales, los algoritmos de suma de comprobación de 16 bits se consideran débiles (es decir, propensos a colisiones de hash), pero para la retroprogramación de 8 bits, crc-16 y Fletcher-16 deberían ser más que suficientes en la

mayoría de los casos. Recientemente se ha mostrado la posibilidad de crear bits sin sentido, capaces de recrear la suma de verificación generada por el algoritmo MD5. Esto significa que cualquier intruso interno o externo podría ser capaz de sustituir contenido digital valioso por datos sin sentido, sin que el sistema de chequeo contra errores pueda detectarlo: el ataque solo se constatará durante la eventual consulta del fichero corrupto. E

El algoritmo de reducción criptográfica MD5, aunque todavía útil a efectos de transmisión de datos, utiliza solo 128 bits y por ello no es recomendable su uso cuando la seguridad sea un factor capital. SHA-1 es ha resultado ser vulnerable en teoría. La longitud de SHA-1 es de 160bits. SHA-2 dispone de diferentes versiones con longitudes de 224, 256, 384 y 512 bits, y resulta algorítmicamente similar a SHA-1. El constante crecimiento de la potencia computacional significa que, a largo plazo, la seguridad aportada por estos algoritmos y sus sumas de verificación también estará en riesgo.

La suma de verificación continúa siendo una opción válida para la detección de errores accidentales, y si se incorpora a un repositorio digital de confianza puede ser perfectamente capaz de desvelar daños intencionados sobre ficheros de datos en escenarios de bajo riesgo. Sin embargo, allí donde exista riesgo, e incluso donde no se contemple, la supervisión de las sumas de verificación y de su viabilidad debe formar parte del plan de preservación.

```
$ md5sum quijote.txt  
  
5ec9ea693fafdad2379b838f392b631e  quijote.txt  
  
$ sha256sum quijote.txt  
5f110a42988f80e119de8aec4a0a1ec399e53c60b7027b63996c3e0458192f95  quijote.txt  
  
$ sha512sum quijote.txt  
5a3bfb9f283f940c9d38381eccb265a06f0e18a0ca4fe0b699d8500542  
59cca0abdad5cfb8d94c70dbd2e95def0298bb4042f5bc764e8d9b223f61  
d1e1fc2de  quijote.txt
```

Uno de los algoritmos más populares es el CRC32. Éste devuelve un valor de comprobación de redundancia cíclica de 32 bits (CRC32). Se utiliza CRC32 para encontrar errores de transmisión de datos y también se puede utilizar CRC32 para comprobar que los datos almacenados en un archivo no se han modificado. Algunos temas a tener en cuenta en su uso son:

- si se utiliza CRC32 para realizar una comprobación de redundancia de datos en modo ASCII y modo Unicode, el Servicio de integración de datos podría generar resultados diferentes para el mismo valor de entrada.
- Si se utiliza CRC32 para realizar una comprobación de redundancia de datos en diferentes sistemas operativos, el Servicio de integración de datos podría generar resultados diferentes para el mismo valor de entrada.

10.3.2. Sumas de verificación en HDFS

HDFS utiliza la suma de comprobación para la integridad de los datos. Una suma de control es un dato de pequeño tamaño derivado de un bloque de datos digitales con el fin de detectar errores. HDFS calcula/computa las sumas de comprobación para cada bloque de datos y finalmente las almacena en un archivo oculto separado en el mismo espacio de nombres de HDFS[?, Voh16]. HDFS utiliza la comprobación de redundancia cíclica de 32 bits (CRC32) como algoritmo de suma de comprobación por defecto, ya que tiene una longitud de 4 bytes y una sobrecarga de almacenamiento inferior al 1 %. Del mismo modo, verifica las sumas de comprobación mientras lee los datos de los nodos de datos. Si hay una discrepancia o error en el valor de la suma de comprobación, la excepción `ChecksumException` será lanzada al cliente durante la recuperación de los datos para su procesamiento. El valor por defecto es de 512 bytes.

```
hadoop fs -copyFromLocal ~/Desktop/dtlScaleData/attr.txt
/tmp/hadoop-name/dfs/data/attr2.txt

13/03/15 15:02:51 INFO util.NativeCodeLoader: Loaded the native-hadoop library
13/03/15 15:02:51 INFO fs.FSInputChecker: Found checksum error: b[0, 0]=
org.apache.hadoop.fs.ChecksumException: Checksum error: /home/name/attr.txt at 0
at org.apache.hadoop.fs.ChecksumFileSystem.$ChecksumFSInputChecker.
readChunk(ChecksumFileSystem.java:219)
at org.apache.hadoop.fs.FSInputChecker.readChecksumChunk(FSInputChecker.java:237)
    at org.apache.hadoop.fs.FSInputChecker.read1(FSInputChecker.java:189)
    at org.apache.hadoop.fs.FSInputChecker.read(FSInputChecker.java:158)
    at java.io.DataInputStream.read(DataInputStream.java:100)
    at org.apache.hadoop.io.IOUtils.copyBytes(IOUtils.java:68)
    ...
    at org.apache.hadoop.fs.FileUtil.copy(FileUtil.java:176)
    at org.apache.hadoop.fs.FileSystem.copyFromLocalFile(FileSystem.java:1183)
    at org.apache.hadoop.fs.FsShell.copyFromLocal(FsShell.java:130)
    at org.apache.hadoop.fs.FsShell.run(FsShell.java:1762)
    at org.apache.hadoop.util.ToolRunner.run(ToolRunner.java:65)
    at org.apache.hadoop.util.ToolRunner.run(ToolRunner.java:79)
    at org.apache.hadoop.fs.FsShell.main(FsShell.java:1895)
copyFromLocal: Checksum error: /home/name/attr.txt at 0
```

Una vez que los bloques de datos son recibidos por los nodos de datos en el clúster de múltiples nodos, calculan y almacenan la suma de comprobación también antes de almacenar los datos en el disco.

Esta suma de verificación se comparará con la almacenada en los nodos de datos mientras el cliente lee los datos. Además, cada nodo de datos individual mantiene un registro persistente de la verificación de la suma de comprobación para llevar la cuenta de cuándo se produjo la última verificación en cada bloque de datos. Se puede desactivar la verificación de la suma de comprobación al enviar un trabajo al clúster. Usando Apache Hadoop 3.1, la comparación de las sumas de comprobación de un archivo almacenado en HDFS se puede hacer con el archivo almacenado localmente.

10.4. Movimientos de datos con DistCP

DistCp (*distributed copy*) es una herramienta utilizada para realizar grandes copias inter/intra-clúster. Con este objetivo utiliza MapReduce para llevar a cabo la distribución, la gestión y recuperación de errores y la elaboración de informes. Su funcionamiento básico consiste en expandir una lista de archivos y directorios en la entrada a las tareas *map*, cada una de las cuales copiará una partición de los archivos especificados en la lista de origen[FHR⁺18].

10.4.1. Arquitectura

Los componentes del nuevo DistCp son los siguientes:

- **DistCp Driver:** Este componente es el responsables de las siguientes tareas:
 - Analizar los argumentos pasados a DistCp desde la línea de comandos.
 - Ejecutar `distcp` tras empaquetar los argumentos del comando en un objeto `DistCpOptions` apropiado. Dentro de estas opciones están las rutas de origen, la ubicación del destino, las opciones de copia (actualización, sobrescritura, etc.).
 - Coordinar la operación de copia mediante la invocación del Copy-listing generator, configurar y lanzar el trabajo MapReduce correspondiente,
- **Copy-listing Generator:** Las clases generadoras de listas de copias son responsables de crear la lista de archivos/directorios a copiar desde el origen. Examinan el contenido de las rutas de origen y registran todas las rutas que necesitan ser copiadas en un archivo secuencial (`SequenceFile`), para ser consumido por el proceso DistCp de Hadoop.
- **InputFormats/MapReduce:** Los componentes `InputFormats` y `MapReduce` son responsables de la copia real de los archivos y directorios desde la ruta de origen a la de destino. El archivo de listado creado durante la generación del listado de copias se consume en este punto, cuando se realiza la copia.

Por defecto, DistCp intenta dimensionar cada mapa de forma comparable para que cada uno copie aproximadamente el mismo número de bytes. Además, las versiones más recientes de DistCp también proporciona una estrategia para dimensionar "dinámicamente" los mapas, permitiendo que los nodos de datos más rápidos copien más bytes que los nodos más lentos. Utilizando la estrategia dinámica en lugar de asignar un conjunto fijo de archivos fuente a cada tarea de mapeo, los archivos se dividen en varios conjuntos. El número de conjuntos supera el número de mapas, normalmente por un factor de 2-3. Cada mapa recoge y copia todos los archivos que figuran en un *chunk*. Cuando se agota un *chunk*, se adquiere y procesa un nuevo chunk, hasta que no quedan más *chunks*.

Al no asignar una ruta de origen a un mapa fijo, las tareas de mapa más rápidas (es decir, los nodos de datos) pueden consumir más bloques, y por tanto copiar más datos, que los nodos más lentos. Aunque esta distribución no es uniforme, es justa con respecto a la capacidad de cada mapeador.

10.4.2. Funcionamiento básico

DistCp utiliza trabajos MapReduce para copiar archivos entre clusters, o dentro del mismo cluster en paralelo. Esto implica dos pasos:

1. Construir la lista de archivos a copiar (conocida como lista de copias)
2. Ejecutar un trabajo MapReduce para copiar archivos, con la lista de copias como entrada

El trabajo MapReduce que realiza la copia sólo tiene mapeadores: cada mapeador copia un subconjunto de archivos en la lista de copias. Por defecto, la lista de copias es una lista completa de todos los archivos en los parámetros del directorio de origen de DistCp. Este proceso puede llevar mucho tiempo, especialmente cuando se trata de un directorio grande y/o muchos subdirectorios.

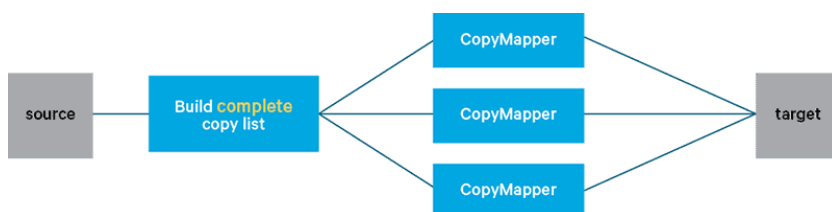


Figura 10.3: Procedimiento básico de copia. Fuente: Cloudera

10.4.3. Funcionamiento Mejorado

Una de las posibles mejoras que se pueden implementar sobre el funcionamiento básico de Distcp es el no copiar los archivos renombrados, es decir, si un archivo se copió en un ciclo de copia de seguridad anterior y luego se renombró, no es necesario volver a copiarlo. Con este fin se utilizan las *snapshots* o instantáneas.

Las instantáneas de HDFS son copias puntuales de sólo lectura de HDFS. Se tarda un tiempo lineal en crear una instantánea y se pueden hacer múltiples instantáneas de forma simultánea. Además se cuenta con la posibilidad de obtener un informe de diferencia de instantáneas ("diff") entre dos instantáneas. Un informe típico de diferencia de instantáneas incluye todas las diferencias entre dos instantáneas, y representa las diferencias con cuatro tipos de operaciones, cuyos operandos son archivos o directorios:

- **CREATE**: archivos/directorios recién creados
- **DELETE**: archivos/directorios eliminados
- **RENOMBRAR**: archivos/directorios renombrados
- **MODIFICAR**: archivos/directorios modificados

Por ejemplo, a partir de dos instantáneas del directorio de origen *s1* y *s2*, donde *s1* se ha copiado en el último ciclo de copia de seguridad, al intentar copiar *s2* se ejecutarán los siguientes pasos:

1. Genera un informe de diferencia de instantáneas entre la instantánea *s1* y la *s2*
2. Aplica las operaciones **RENAME** y **DELETE** en el informe de diferencia de instantáneas al sistema de archivos de destino. Este paso se denomina paso de sincronización.
3. Ejecuta los pasos predeterminados de DistCp para copiar de *s2* al sistema de archivos de destino
4. Después del paso de sincronización, los mapeadores de DistCp en el paso 3 encuentran que los archivos renombrados son los mismos entre el origen y el destino, y no los copia de nuevo.

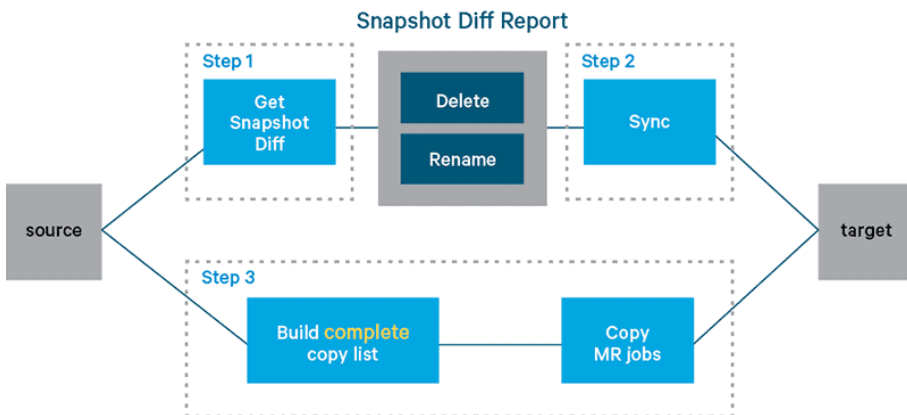


Figura 10.4: Procedimiento mejorado de copia. Fuente: Cloudera

10.4.4. Uso

La invocación más común de DistCp es una copia entre clusters:

```
hadoop distcp hdfs://nn1:8020/foo/bar hdfs://nn2:8020/bar/foo
```

Esto expandirá el espacio de nombres bajo /foo/bar en nn1 en un archivo temporal, dividirá su contenido entre un conjunto de tareas de mapeo e iniciará una copia en cada NodeManager de nn1 a nn2. La opción -update se utiliza para copiar archivos del origen que no existen en el destino o que difieren de la versión de destino. -overwrite sobrescribe los archivos de destino que existen en el destino. Por ejemplo, si contamos con esta estructura de directorios en nuestro origen:

```
hdfs://nn1:8020/source/first/1
hdfs://nn1:8020/source/first/2
hdfs://nn1:8020/source/second/10
hdfs://nn1:8020/source/second/20
```

Al invocar sin utilizar -update o -overwrite Distcp va a crear los directorios first/ y second/ de la siguiente forma:

```
distcp hdfs://nn1:8020/source/first hdfs://nn1:8020/source/second
hdfs://nn2:8020/target
```

```
hdfs://nn2:8020/target/first/1
hdfs://nn2:8020/target/first/2
hdfs://nn2:8020/target/second/10
hdfs://nn2:8020/target/second/20
```

Cuando se especifica -update o -overwrite, se copia el contenido de los directorios de origen en el destino, y no los propios directorios de origen. Por lo tanto, se generarán los siguientes contenidos en /target:

```
distcp -update
hdfs://nn1:8020/source/first hdfs://nn1:8020/source/second
hdfs://nn2:8020/target
```

```
hdfs://nn2:8020/target/1
hdfs://nn2:8020/target/2
hdfs://nn2:8020/target/10
hdfs://nn2:8020/target/20
```

La opción -diff sincroniza los archivos de un clúster de origen a un clúster de destino con un snapshot diff.

```
hadoop distcp -update -diff snap1 snap2 /src/ /dst/
```


Bibliografía

- [AGN17] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. Data profiling: A tutorial. In *Proceedings of the 2017 ACM International Conference on Management of Data*, pages 1747–1751, 2017.
- [CZ15] Li Cai and Yangyong Zhu. The challenges of data quality and data quality assessment in the big data era. *Data science journal*, 14, 2015.
- [FHR⁺18] Heitor Faria, Rodrigo Hagstrom, Marco Reis, Breno GS Costa, Edward Ribeiro, Maristela Holanda, Priscila Solis Barreto, and Aletéia PF Araújo. A hadoop open source backup software solution. 2018.
- [LEHO16] Imane Lebdaoui, Said El Hajji, and Ghizlane Orhanou. Managing big data integrity. In *2016 International Conference on Engineering & MIS (ICEMIS)*, pages 1–6. IEEE, 2016.
- [PA15] Mohan V Pawar and J Anuradha. Network security and types of attacks in network. *Procedia Computer Science*, 48:503–506, 2015.
- [PP15] Pradeep Pasupuleti and Beulah Salome Purra. *Data lake development with big data*. Packt Publishing Ltd, 2015.
- [SJR15] Sithu D Sudarsan, Raoul P Jetley, and Srini Ramaswamy. Security and privacy of big data. In *Big Data*, pages 121–136. Springer, 2015.
- [Voh16] Deepak Vohra. *Practical Hadoop ecosystem: a definitive guide to hadoop-related frameworks and tools*. Apress, 2016.
- [WZL06] Richard Y Wang, Mostapha Ziad, and Yang W Lee. *Data quality*, volume 23. Springer Science & Business Media, 2006.